

Cancer response predictor - pipeline modules & data flow (branch merge/as). Each box: NAME (file) . key options . IN / OUT. Target y in {0,1}: 1 = responds, 0 = does not. Rule: OUT of each module = IN of the next.

GUI - app.py + ui/ Password gate AND .tml4model key gate run first, both fail closed. Two pages: 'Build & predict' (tabs 1 Build / 2 Save-load / 3 Predict) and 'User manual' (/manual). Help & FAQs folds at the foot of both.

1 LOADER (m1_loader.py + cohorts.py)
sources: your CSV/TSV/TXT . bundled IMvigor210 cohort (298 patients, 68 responders, +TMB for 234) . Tan layout (transposed) . synthetic demo
caps: 50 MB per table, 50M cells (TML4_MAX_CELLS overrides)
IN: a table - rows = patients, cols = genes + a last label column
OUT: X [n x g], y [n in {0,1}], feature names

1b IMAGING LOADER (m1b_imaging_loader.py + imaging/)
a SIBLING of module 1. not a later stage - same output contract
zip or folder //, up to 500 MB
reads png / tif / jpg / bmp / webp (multi-page TIFF = a 3-D volume)
measures ~102 radiomic features: intensity, texture, gradient, shape, radial
ONE ROW PER PATIENT always (a patient's images are aggregated) - not an option, it is what stops the same patient landing in train and test
OUT: X [n x ~102], y [n], feature names

X, y — FIT ON ALL DATA — X, y (same contract)

2 PREPROCESS (m2_preprocess.py) 5 options
identity / z-score / robust (median-IQR) / variance filter + z-score / log2 + z-score
IN: X [n x g] (from module 1 or 1b)
OUT: X' [n x g] - same patients, columns rescaled / filtered

AUTO (pick best)

Set any of stages 2-4 to 'Auto'; Build evaluates those options in parallel (slowest-first), skips combos that can't fit, never picks an unreliable one, keeps the best by BALANCED score.

Optional TIME BUDGET drops setups predicted to overrun before they run (default: no limit = full search).

COMPARE grid (2 x 3 x 4)
-> table ranked your way (balanced / accuracy / sensitivity / specificity / AUC) + CSV and PDF.

ASSEMBLER (assemble.py)

build_machine(2 -> 3 -> 4)
= ONE sklearn Pipeline.

Learned scaling + feature-map basis travel WITH the classifier, so predict re-applies the exact same steps. Raw X in -> y_hat / P out.

For imaging, the MEASUREMENT settings travel too - a scan scored later is measured the way the training cohort was.

+ compatibility_warnings():
flags combos that can't fit.

3 FEATURE MAP (m3_feature_map.py) 5 options
identity / PCA(50) / class-conditional PCA / rank (TSP-style) / select best 500
IN: X' [n x g] (from module 2)
OUT: Z [n x k] - model-ready feature matrix

4 CLASSIFIER (m4_ml.py + estimators.py) 11 options
RF . GB . Hist GB . LogReg . SVM linear . SVM RBF . decision tree . naive Bayes
k-NN . K-TSP (top-scoring pairs) . nearest shrunken centroid
the SVMs decide the class by their OWN rule; calibration only reports P
IN: Z [n x k] + y [n] (from module 3)
OUT: fitted classifier -> y_hat + P(respond) per patient

CROSS-VALIDATE

5 EVALUATE (m5_evaluate.py)
k-fold CV (auto or manual folds); tiny classes -> no-fold resubstitution
IN: assembled machine + X, y (from modules 2-4)
OUT: metrics = accuracy, sensitivity, specificity, balanced, AUC, confusion
UNRELIABLE if ANY of four: predicts one class / finds under 10% of a group / does not beat chance on BALANCED accuracy / the solver never converged

METRICS

6 REPORT (m6_report.py)
verdict-first metrics, confusion matrix, run config; built in memory
IN: metrics + run config (from module 5)
OUT: on-screen report + downloadable PDF

TRAINED MACHINE (the assembled Pipeline, fit on ALL data)
kept in the session; saved / loaded as a locked .tml4model file

.tml4model

PREDICT (ui/predict.py)
IN: trained machine + new patients - pasted, a table, or scans (measured with the settings stored in the model)
checks the feature count; tolerates ONE trailing 0/1 marker column
OUT: yes/no + P(respond) per patient

PREDICTION

7 EXPLAIN (m7_explain.py)
occlude one feature at a time, watch predict_proba shift
IN: trained machine + patient x + training-set means (occlusion ref)
OUT: top features behind that prediction (gene-level even under PCA)

Prediction & comparison PDFs are produced by m6_report.py too:
per-patient verdicts + top features, and setup-comparison tables.

SAVE / LOAD (ui/model.py)

Download the trained machine as a LOCKED .tml4model file:
authenticated-encrypted + app-locked - opens only here, can't be edited or reused (see README Security).

Needs TML4_MODEL_KEY;
no key, no start.